

Tasks

Task 1: BMI Calculator & Category

Concept Focus: *if-else-if-else, arithmetic expressions*

Write a C program that reads a person's weight (kg) and height (m), computes their Body Mass Index ($BMI = \text{weight} / \text{height}^2$), and prints the corresponding category using an if-else-if-else ladder: Underweight (< 18.5), Normal (18.5-24.9), Overweight (25-29.9), Obese (≥ 30).

Hints:

- Compute BMI as a float/double and print it rounded to two decimal places.
- Order the comparisons from the lowest range to the highest, or vice versa, and keep the boundaries consistent.

Sample Input / Output:

```
Enter weight (kg): 68
Enter height (m): 1.72
BMI = 22.99 -> Category: Normal
```

Task 2: Triangle Type Classifier

Concept Focus: *if-else-if-else, logical operators*

Write a C program that reads the lengths of three sides of a triangle. First validate that the sides can actually form a triangle (the sum of any two sides must be greater than the third), then classify it as Equilateral, Isosceles, or Scalene.

Hints:

- The validity check needs three conditions joined with `&&` (each pair-sum must exceed the remaining side).
- If the sides are invalid, print a message and skip the classification.
- Check for three equal sides first, then two equal sides, before falling back to scalene.

Sample Input / Output:

```
Enter three side lengths: 5 5 8
Valid triangle -> Type: Isosceles
```

Task 3: ATM Cash Withdrawal Simulator

Concept Focus: *if-else, logical operators (&&, ||)*

Write a C program that simulates a simple ATM withdrawal. Read the account balance and the amount the user wants to withdraw. The withdrawal should only succeed if the amount is a multiple of 500, does not exceed the available balance, and does not exceed a daily limit of 25,000. Use logical operators to combine all the checks and print an appropriate message for each possible failure reason.

Hints:

- Use the modulus operator to check that the amount is a multiple of 500.
- Combine the three conditions with `&&` for the success case, but consider giving a distinct message for each failing condition rather than one generic error.

Sample Input / Output:

```
Enter balance: 15000
Enter withdrawal amount: 4500
Withdrawal successful. Remaining balance: 10500
```

Task 4: Vowel or Consonant

Concept Focus: *switch statement, character input*

Write a C program that reads a single alphabet character and uses a switch statement to determine whether it is a Vowel or a Consonant. The program should work for both uppercase and lowercase input and should print an error message if the input is not a letter.

Hints:

- Convert the character to lowercase (or handle both cases in the switch) before checking.
- List a, e, i, o, u as separate case labels stacked together (fall-through) that all print 'Vowel'.
- Use the default case for consonants, but first validate the character is a letter using `isalpha()` from `<ctype.h>` before entering the switch.

Sample Input / Output:

```
Enter a character: E
E is a Vowel
```

Task 5: Electricity Bill Calculator

Concept Focus: *if-else-if-else, slab-based computation*

Write a C program that reads the number of electricity units consumed in a month and computes the bill using the following slab rates, implemented with an if-else-if-else ladder: first 100 units @ Rs. 5/unit, next 100 units (101-200) @ Rs. 8/unit, next 200 units (201-400) @ Rs. 12/unit, and any units above 400 @ Rs. 15/unit.

Hints:

- Decide whether each slab rate applies only to that slab's units or to the whole consumption once a threshold is crossed - a true slab system charges each portion at its own rate.
- Test your program with values that fall exactly on a boundary (e.g., 100, 200, 400 units) to make sure no units are double-counted or skipped.

Sample Input / Output:

```
Enter units consumed: 250
Total Bill: Rs. 3300
```

Task 6: Menu-Driven Number Utility

Concept Focus: *switch statement, nested if-else*

Write a C program that displays a menu with the options: (1) Check Even/Odd, (2) Check Prime, (3) Find Square, (4) Exit. Read the user's choice and a number where needed, then use a switch statement to route to the correct operation. For option 2, use a loop or nested if-else logic to determine whether the number is prime.

Hints:

- Keep the menu and the switch inside a loop (e.g., do-while) so the user can perform multiple operations until choosing Exit.
- For the prime check, remember 0, 1, and negative numbers are not prime, and you only need to test divisors up to the square root of the number.
- Use a default case in the switch to catch invalid menu choices.

Sample Input / Output:

```
1. Even/Odd 2. Prime 3. Square 4. Exit
Enter choice: 2
Enter a number: 29
29 is Prime
```